

Konzeption und Implementierung eines GitHub-basierten Branching-, Review- und Projektmanagementprozesses mit KI-gestützter Code-Compliance

T4_2000

Studiengang Elektrotechnik

Duale Hochschule Baden-Württemberg Ravensburg
Campus Friedrichshafen

Florian Germann

Eingereicht am: 21.05.2026

Matrikelnummer, Kurs: 7582635, TEA24

Unternehmen: fpt Systems GmbH, 88279, Amtzell

Betreuer im Unternehmen: Simon Würzer

Inhalt

Erklärung zum Einsatz von KI-gestützten Werkzeugen	1
1 Einleitung	3
2 Ausgangslage und Zielbild	5
2.1 Problematik	5
2.1.1 Branching	5
2.1.2 Management	5
2.1.3 Code-Review	6
2.1.4 Entwicklungsumgebung	6
2.2 Lösungsansatz	6
2.2.1 Branching	6
2.2.2 Management	7
2.2.3 Code-Review	7
2.2.4 Entwicklungsumgebung	7
2.2.5 Fazit	7
3 Grundlagen und Ausgangslage	9
3.1 Industriestandards	9
3.1.1 GitHub-Flow	9
3.1.2 Git-Flow	11
3.2 Richtlinien E-SW	13
3.3 Zwischenfazit zur Branching-Wahl	15
4 Evaluation des Code-Review-Ansatzes	16
4.1 Rahmenbedingungen und Setup bei fpt	16
4.2 Testablauf	17
4.3 Ergebnisse und Problemanalyse	18
4.3.1 Grundproblem	18
4.3.2 Problem Kommentarunterdrückung	19
4.4 Feedback und weitere Verbesserung	22
5 Pilotierung im Projekt	23
5.1 Umsetzung im Pilotprojekt	24
5.1.1 Branching Strategien	24
5.1.2 Copilot Code-Review	24

6 Projektmanagementprozess mit ZenHub	25
6.1 Aktueller Stand bei E-SW	26
6.2 Übertragung auf die O-SW	26
7 Schulung und Wissenssicherung	28
7.1 Aktuelles Wissen der Mitarbeiter	28
7.2 Wahl der Schulungsumgebung	29
7.2.1 Microsoft Learn	29
7.2.2 GitHub Skills	29
7.2.3 YouTube	29
7.2.4 Eigene Dokumentation	30
7.3 Umsetzung der Dokumentation	30
7.3.1 Git und GitHub Grundlagen	30
7.3.2 Branching Strategien	30
8 Fazit	32
A Literatur	33
B Glossar	34



Erklärung zum Einsatz von KI-gestützten Werkzeugen

Im Rahmen dieser Projektarbeit wurden KI-gestützte Werkzeuge ausschließlich als unterstützende Hilfsmittel eingesetzt. Die folgende Tabelle gibt einen Überblick über die genutzten Systeme und deren konkreten Verwendungszweck:

Werkzeug	Anbieter	Verwendungszweck
GitHub Copilot	Microsoft / GitHub	Automatisiertes Code-Review anhand der Coding-Conventions bei Pull Requests
GitHub Copilot Chat	Microsoft / GitHub	Analyse von Review-Problemen, Recherche zu Lösungsansätzen, Überprüfung von Rechtschreibung
GitHub Copilot in Visual Studio Code	Microsoft / GitHub	Rechtschreib und Grammatikprüfung, Ausführen lokaler Agents
GitHub Copilot Agent	Microsoft / GitHub	Tiefergehende Repository-Analyse zur Ursachenforschung bei Kommentarunterdrückung
Microsoft Copilot „Prompt Coach,,	Microsoft	Unterstützung bei der KI-kompatiblen Überarbeitung und Präzisierung der Coding-Conventions

Tabelle 1 – Eingesetzte KI-Werkzeuge

Alle durch KI-Werkzeuge generierten Ausgaben wurden vom Autor inhaltlich geprüft, bewertet und erst nach kritischer Sichtung übernommen. Eine automatische oder ungeprüfte Übernahme von KI-Inhalten erfolgte nicht. Die inhaltliche Verantwortung für diese Arbeit liegt vollständig beim Autor.

KI-Werkzeuge wurden nicht zur Erstellung des Fließtextes dieser Arbeit eingesetzt, sondern ausschließlich zur Prüfung auf Rechtschreibfehler.



Einleitung

Eine strukturierte Quellcodeverwaltung ist eine zentrale Voraussetzung für moderne Softwareentwicklung. Gerade in verteilten Teams, die gleichzeitig an mehreren Komponenten arbeiten, muss jederzeit nachvollziehbar sein, welche Änderungen wann und warum erfolgt sind. Git-basierte Plattformen wie GitHub bieten dafür die technische Grundlage.

GitHub ermöglicht die zentrale Ablage, Verwaltung und gemeinsame Weiterentwicklung von Softwareprojekten. Durch die zugrunde liegende Git-Versionskontrolle lassen sich Änderungen transparent dokumentieren und frühere Zustände bei Bedarf wiederherstellen. Besonders in komplexen Projekten erhöht das die Nachvollziehbarkeit und reduziert Abstimmungsaufwand.

Ein wesentliches Konzept in diesem Zusammenhang sind Branches. Sie erlauben es, neue Funktionen, Fehlerbehebungen oder Experimente getrennt von der Hauptlinie umzusetzen. Erst nach einer Prüfung werden Änderungen zurückgeführt. Dadurch können mehrere Personen parallel arbeiten, ohne sich gegenseitig zu blockieren.

Die Kombination aus Versionskontrolle, klarer Historie und parallelen Entwicklungszweigen schafft damit eine belastbare Grundlage für Teamarbeit in der Softwareentwicklung.

Wie gut GitHub in der Praxis funktioniert, hängt jedoch stark von den gewählten Prozessen ab. Insbesondere eine passende Branching-Strategie und ein funktionierendes Team-Management sind entscheidend. Branching-Regeln legen fest, wie Features, Bugfixes und Releases vorbereitet und integriert werden. Sie beeinflussen damit nicht nur den technischen Ablauf, sondern auch Koordination, Planbarkeit und Qualitätssicherung.

Gleichzeitig reicht eine gute Branching-Strategie allein nicht aus. Ebenso wichtig ist eine strukturierte Aufgabenplanung im Team. Projektmanagement-Werkzeuge helfen dabei, Arbeitspakete transparent zu priorisieren, Verantwortlichkeiten eindeutig zuzuordnen und Fortschritte laufend sichtbar zu machen. Durch die enge Verzahnung mit Entwicklungsprozessen, automatisierten Workflows und übersichtlichen Boards entsteht ein gemeinsames Verständnis darüber, welche Aufgaben als Nächstes anstehen und wo Engpässe drohen. Das stärkt Zusammenarbeit und erhöht zugleich Qualität und Umsetzungsgeschwindigkeit.

Zur didaktisch klaren Einordnung folgt die Arbeit der Abfolge Ausgangslage, konzeptioneller Lösungsansatz, fachliche Grundlagen, Evaluation im Testkontext, Pilotierung im Projekt, organisatorische Implementierung und Schulung sowie abschließendes Fazit.



Ausgangslage und Zielbild

2.1 Problematik

2.1.1 Branching

In der Abteilung [O-SW](#) gibt es bislang keine einheitliche und verbindliche Vorgehensweise für den Umgang mit GitHub-Branche. Zwar wurden Richtlinien aus der [E-SW](#) bereits für interne Bibliotheken und Template-Projekte übernommen, in der praktischen Arbeit an Anlagen zeigt sich jedoch, dass diese Vorgaben nicht in allen Fällen direkt passen. Die Entwicklungsabläufe unterscheiden sich in Struktur und Inhalt, wodurch im Alltag Unklarheiten entstehen.

Fehlt diese Passgenauigkeit, werden Branching-Entscheidungen häufig individuell getroffen. Das führt zu uneinheitlichen Abläufen, erschwert die Nachvollziehbarkeit von Änderungen und erhöht das Konfliktpotenzial beim Zusammenführen von Code. Gleichzeitig sinkt die Effizienz in der Zusammenarbeit, weil gemeinsame Regeln nicht konsequent angewendet werden können.

2.1.2 Management

Im Bereich Management besteht in der [O-SW](#) derzeit das Problem, dass keine dedizierte Team-Management-Software genutzt wird. Die Release-Planung erfolgt überwiegend über mündliche Absprachen und E-Mail-Verläufe. Das ist in der tägli-

chen Praxis fehleranfällig und nur eingeschränkt transparent. Zusätzlich entstehen regelmäßig Engpässe, wenn neue Versionen von [FlexOp](#) kurzfristig bereitgestellt werden und notwendige Vorbereitungen nicht rechtzeitig angestoßen wurden.

2.1.3 Code-Review

Trotz klar formulierter Coding-Conventions gelangen weiterhin falsch benannte Variablen oder unzureichend dokumentierte Codeanteile in Projekte. Ursache ist vor allem, dass die Einhaltung der Regeln bisher nur eingeschränkt und überwiegend manuell geprüft wird. Für [TwinCat](#) stehen keine klassischen Linter¹ oder vergleichbare Standardwerkzeuge zur Verfügung, wodurch die automatisierte Qualitätskontrolle deutlich erschwert wird.

2.1.4 Entwicklungsumgebung

Im Moment existiert in der [O-SW](#) das Problem, dass keine einheitliche Entwicklungsumgebung vorhanden ist. Das kommt daher, da die Lizenzen für die alte Lösung ausgelaufen sind und zum einen wegen Kosten und zum anderen wegen langen Update Zeiten nun eine neue Lösung gefunden werden soll. Allerdings steht zum aktuellen Zeitpunkt noch kein neues Programm bereit um das alte zu ersetzen.

Aus diesen vier Problemfeldern ergibt sich der Bedarf nach einem gemeinsamen, technisch umsetzbaren und von der gesamten Abteilung getragenen Lösungsansatz.

2.2 Lösungsansatz

2.2.1 Branching

In der [E-SW](#) sind bereits funktionierende Branching-Strategien etabliert. Diese werden in der [O-SW](#) heute schon bei internen Bibliotheken und Template-Projekten eingesetzt und sollen im nächsten Schritt auf die Anlagenentwicklung ausgeweitet werden.

Dazu werden die bestehenden Regeln systematisch darauf geprüft, inwieweit sie zu den spezifischen Abläufen der [O-SW](#) passen. Ziel ist eine konsistente Richtlinie, die fachlich tragfähig ist und unternehmensweit einheitlich angewendet werden kann.

¹Ein Linter prüft Code automatisch auf Fehler und Stilabweichungen.

2.2.2 Management

Für die aktuell wenig strukturierten Management-Abläufe in der O-SW liegt innerhalb der [fpt Systems GmbH](#) bereits ein erprobter Lösungsweg vor: In der E-SW wird seit einiger Zeit die Team-Management-Software ZenHub genutzt. Damit werden Aufgaben priorisiert, Teammitgliedern zugewiesen und Releases von [FlexOp](#) sowie weiteren Hochsprachen-Projekten geplant.

ZenHub soll deshalb auch in der O-SW eingeführt werden. Ziel ist es, die Weiterentwicklung der Standardbibliotheken und die Pflege der Template-Projekte für die SPS-Programmierung besser zu strukturieren. Gleichzeitig soll die Release-Abstimmung mit der E-SW verlässlicher werden.

2.2.3 Code-Review

Zur Verbesserung der Code-Qualität wird ein flexibler Ansatz verfolgt, der klassische Linter ergänzungsweise in Teilbereichen ersetzt. In Abstimmung mit dem KI-Experten der E-SW Ralf Scheyerle und dem in der O-SW für Richtlinien verantwortlichen Mitarbeiter Robin Hettel wurde daher die Einführung von GitHub-Copilot-gestützten Code-Reviews vorgeschlagen. Dieser Ansatz lässt sich an projektspezifische Anforderungen anpassen und unterstützt eine effizientere Qualitätssicherung.

2.2.4 Entwicklungsumgebung

Dass die Entwickler in der O-SW wieder nahtlos programmieren können soll nun eine neue Lösung gefunden werden. Bisher wurde hier Visual Studio 2017 in Kombination mit dem Explorer, Github Desktop oder Sourcetree und ein in Visual Studio integriertes Programm für KUKA Steuerungen verwendet. Das integrierte, Firmeneigene Programm namens FlexManager ist allerdings schon sehr alt und kann nicht auf die neue Visual Studio Version geupdatet werden. Da aus diesem Grund ohnehin eine Neuentwicklung nötig wäre und die Lizenzen für Visual Studio auslaufen, wurde beschlossen eine Entwicklungsumgebung zu erarbeiten. Hierzu wurde sich für VSCode in Kombination mit der TwinCat XAE Shell entschieden, da VSCode sich durch Extensions hervorragend erweitern lässt und sich so ideal auf die Anforderungen anpassen lässt.

2.2.5 Fazit

Damit die eingeführten Verbesserungen im Alltag zuverlässig funktionieren, müssen alle Mitarbeitenden nach der Implementierung informiert und entsprechend geschult

werden. Zusätzlich ist eine zentrale, leicht zugängliche Dokumentation der neuen Abläufe notwendig, damit die Anwendung langfristig konsistent bleibt.

Insgesamt dienen die Maßnahmen dazu, sowohl die Arbeitsqualität als auch die Code-Qualität in der fpt Systems GmbH nachhaltig zu erhöhen.

3

Grundlagen und Ausgangslage

3.1 Industriestandards

In der Softwareentwicklung haben sich über viele Jahre hinweg unterschiedliche Branching-Standards etabliert. Im Folgenden werden die zwei bekanntesten Modelle vorgestellt und im Hinblick auf ihre Eignung für den vorliegenden Anwendungsfall eingeordnet.

3.1.1 GitHub-Flow

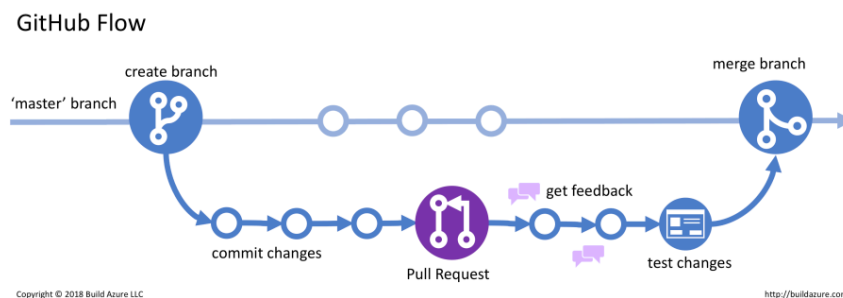


Abbildung 1 – GitHub Flow

GitHub-Flow ist ein schlankes, auf kontinuierliche Auslieferung ausgerichtetes Branching-Modell. Es eignet sich besonders für Teams, die schnell auf neue Anforderungen reagieren und Änderungen zeitnah bereitstellen müssen. Charakteristisch sind kurze Feature-Branched, häufige Pull Requests und automatisierte Tests, sodass neue Anpassungen zügig in den Main-Branch integriert werden können. Entsprechend wird

GitHub-Flow vor allem in DevOps-nahen Umgebungen und Projekten mit hoher Release-Frequenz, etwa bei Web-Anwendungen, häufig eingesetzt.

Die Ausgestaltung Pull-Request-basierter Zusammenarbeit und ihre Auswirkungen auf den Entwicklungsprozess sind in der Literatur ausführlich untersucht worden [1].

Branch	Beschreibung
main(master)	Enthält fertige Releases Stabil - Änderungen nur aus feature/* per PR
feature/*	Entwicklungsbranch für einzelne Features oder Fixes werden direkt von main erstellt nach Fertigstellung direkt PR in main

Tabelle 2 beschreibt die einzelnen Branches und ihre Aufgaben, während Abbildung 1 den Ablauf grafisch ergänzt. Daraus wird ersichtlich, dass diese Strategie vor allem für schnelle, häufige Feature-Releases geeignet ist. Ob sie im konkreten Fall sinnvoll ist, hängt jedoch immer von den organisatorischen und technischen Rahmenbedingungen ab.

VORTEILE

- Schnelle Iteration
- Geringe Komplexität
- Ideal für kontinuierliche Integration

NACHTEILE

- Wenig Struktur für große Releases
- Risiko eines instabilen Main-Branches bei häufigem Mergen

FAZIT

Zusammenfassend ist GitHub-Flow eine einfache und agile Strategie, die besonders gut zu Teams passt, die auf kontinuierliche Integration und kurze Release-Zyklen setzen. Die kurzen Feature-Branche und direkten Mergen in main sorgen für Flexibilität und wenig Prozess-Overhead. Im Kontext der **O-SW** mit stärker meilensteinorientierten Releases zeigt sich allerdings ein Nachteil: Da eine separate Release-Vorbereitung

fehlt, steigt das Risiko von Instabilitäten im Freigabeprozess. Für diesen Kontext sind daher strukturiertere Modelle wie Git-Flow meist besser geeignet.

Damit stößt GitHub-Flow in der O-SW an klare Grenzen. Im nächsten Schritt wird daher Git-Flow betrachtet, um die Eignung für den vorliegenden Kontext zu bewerten.

3.1.2 Git-Flow

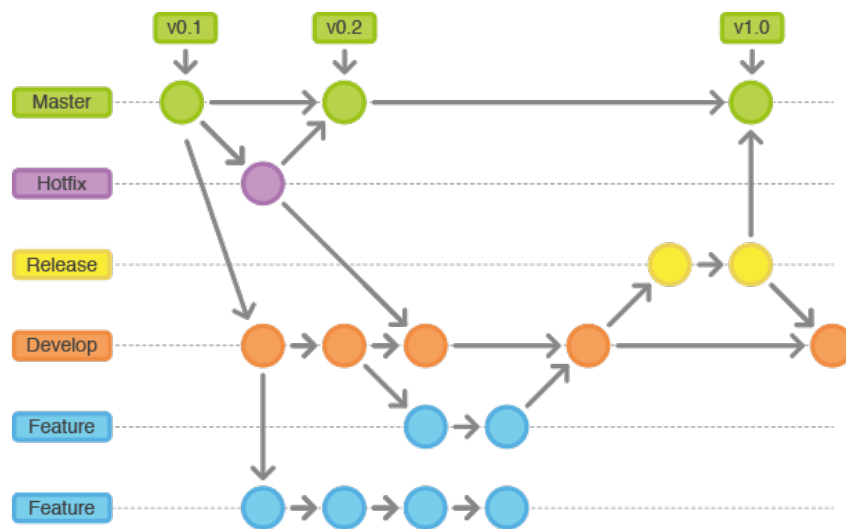


Abbildung 2 – Git Flow

Git-Flow ist ein strukturiertes Branching-Modell für Git, das Entwicklungsabläufe klar trennt und gleichzeitig stabile Releases unterstützt. Das Modell wurde 2010 von Vincent Driessen vorgestellt und wird vor allem in Projekten eingesetzt, in denen mehrere Features, Hotfixes und Versionen parallel entstehen.

Die ursprüngliche Beschreibung des Modells geht auf Driessen zurück [2].

Das Konzept basiert auf mehreren langlebigen Branches mit eindeutig definierten Aufgaben. Abbildung 2 zeigt, wie diese Branches zusammenwirken und wo die eigentliche Entwicklung stattfindet. Ergänzend werden in Tabelle 3 die einzelnen Branches beziehungsweise Branch-Gruppen beschrieben.

Branch	Beschreibung
main(master)	Enthält fertige Releases Stabil - Änderungen nur aus release/* per PR Änderungen in main müssen zurück in develop fließen.
develop	Grundlage aller Entwicklungsbranches Stabil - Änderungen nur als PR
feature / *	Entwicklungsbranches neuer Features In diesen wird gearbeitet Strukturierung über Subbranches möglich
bugfix / *	Entwicklungsbranch für Bugfixes Wird von develop abgespalten um Bugs zu entfernen
release / *	Vorbereitung für den Release Wird von develop abgespalten Änderungen fließen wieder in develop z. B. Erstellung von Release Notes
hotfix / *	Hotfix von main Änderungen fließen auch in develop

Dieses System eignet sich besonders für strukturierte Abläufe mit klar definierten, umfangreicheren und aufeinander aufbauenden Releases. Für Rolling-Release-Ansätze ist es dagegen weniger passend, da die Zeitspanne bis zur Veröffentlichung tendenziell länger ist. Da in der [O-SW](#) Releases primär an Projektmeilensteinen vorgesehen sind, ist Git-Flow für diesen Anwendungsfall gut geeignet.

3.2 Richtlinien E-SW

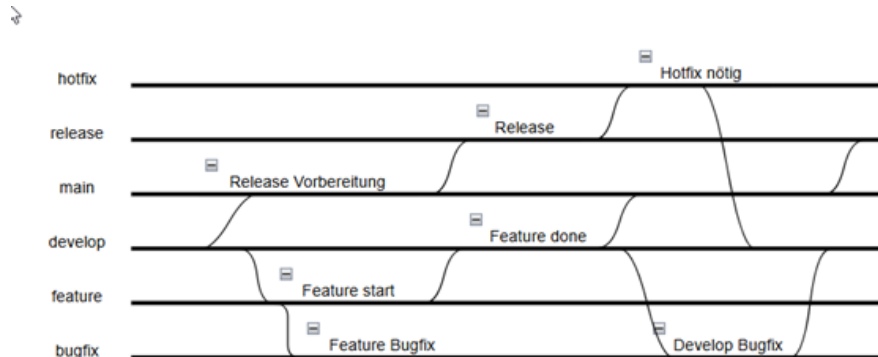


Abbildung 3 – Git Flow E-SW

Zu Beginn wurde der aktuelle Stand der Richtlinien bei Ralf Scheyerle abgefragt, der in der [E-SW](#) für dieses Thema verantwortlich ist. Da das Team nur aus fünf Personen besteht, waren die Vorgaben bislang überwiegend mündlich überliefert und lediglich in einem einfachen Word-Dokument festgehalten.

Die in Abbildung 3 dargestellte Vorgehensweise zeigt, dass die aktuelle Strategie der [E-SW](#) den Git-Flow-Prinzipien aus Abschnitt 3.1.2 weitgehend folgt. Ein wesentlicher Unterschied, der in Tabelle 4 genauer erläutert wird, liegt in der Release-Logik: Releases werden nicht in separaten release/*-Branches vorbereitet, sondern in main durchgeführt und anschließend über einen release/*-Branch veröffentlicht. Dadurch ist der Zugriff auf frühere Release-Stände sehr schnell möglich, ohne über Tags oder Pull Requests nach Versionsnummern suchen zu müssen. Nachteilig ist jedoch, dass sich über die Zeit viele inaktive Branches ansammeln und die Übersichtlichkeit sinkt.

Branch	Beschreibung
main	Für Releasevorbereitung Stabil - Änderungen nur aus „develop“ per PR Änderungen in main müssen zurück in develop fließen.
develop	Grundlage aller Entwicklungsbranches Stabil - Änderungen nur als PR
feature / *	Entwicklungsbranch neuer Features Verknüpfung mit ZenHub Epics über ID möglich Beispiel: feature/Fx.FlexOP-1640_SSI-Packagerstellung SubFeature (Task-Ebene): feature/Fx.FlexOP-1640_SSI-Packagerstellung/FPT.SSI-51_Externe-Steuerungsconfig
bugfix / *	Entwicklungsbranch für Bugfixes Verknüpfung mit ZenHub Bugs über ID möglich Beispiel: bugfix/Fx.FlexOP-1234
release / *	Releases von main Kann auch über Tag gelöst werden, sollte aber als Branch gezogen werden, sobald ein Hotfix nötig wird. Beispiel: release/Fx.FlexOP_v4.5.5.x
hotfix / *	Hotfix von Release-Branch Sollte wieder zurück in develop fließen Verknüpfung mit ZenHub Bug über ID Beispiel: hotfix/Fx.FlexOP_v4.5.5.12_2547

Das Word-Dokument enthält in Abbildung 3 und Tabelle 4 die Kernelemente einer angepassten Git-Flow-Variante. Der zentrale Unterschied besteht darin, dass Releases

nicht in separaten `release/*`-Branches vorbereitet, sondern in `main` erstellt und anschließend über einen `release/*`-Branch veröffentlicht werden. Abgesehen davon entsprechen die grundsätzlichen Vor- und Nachteile weitgehend dem klassischen Git-Flow-Modell.

3.3 Zwischenfazit zur Branching-Wahl

Auf Grundlage der Gespräche mit Robin Hettel wurde festgehalten, dass die Einführung von Git-Flow sowohl fachlich sinnvoll als auch organisatorisch gewünscht ist. Der Hauptgrund liegt darin, dass Git-Flow für wenige, größere Releases besser geeignet ist als GitHub-Flow. Zudem ist der in Abschnitt 3.2 genannte Vorteil für den vorliegenden Kontext weniger entscheidend, da Releases ausschließlich an Projektmeilensteinen stattfinden und der aktuelle, stabile Code-Stand im Vordergrund steht.

4

Evaluation des Code-Review-Ansatzes

GitHub Copilot kann mehr leisten als reine Codevervollständigung. Das System kann Entwickler auch dabei unterstützen, bestehende Coding-Conventions im Alltag konsequenter einzuhalten. Während der Arbeit analysiert Copilot den jeweiligen Kontext, erkennt wiederkehrende Muster und orientiert sich an projektinternen Standards. Dadurch entstehen Vorschläge, die nicht nur funktional sind, sondern auch besser zum Stil des Projekts passen. Teams profitieren dadurch von konsistenterer Codequalität, weniger manuellen Nacharbeiten und einem effizienteren Review-Prozess.

Die Bedeutung strukturierter Code-Reviews für Qualitätssicherung und Teamzusammenarbeit ist in der Forschung breit belegt [3], [4].

4.1 Rahmenbedingungen und Setup bei fpt

Bei fpt befindet sich der Einsatz von Copilot für Code-Reviews derzeit noch in einer Erprobungsphase. Um produktive Bereiche nicht zu beeinflussen, wurde dafür ein separates Test-Repository eingerichtet. Da Copilot bei Reviews nur auf Inhalte des jeweiligen Repositories zugreifen kann, müssen die Coding-Conventions dort lokal im .github-Ordner hinterlegt sein. Dies geschieht über .md-Dateien, die GitHub direkt gerendert darstellt. Damit Änderungen nicht manuell in jedes Repository verteilt werden müssen, wurde das in Abbildung 4 gezeigte Verteilungssystem aufgebaut. Jeder Block steht für ein Repository, die Pfeile repräsentieren GitHub Actions.

Die Funktionsweise von Pull-Request-Reviews und branchbezogenen Schutzmechanismen ist in der offiziellen GitHub-Dokumentation beschrieben [5], [6].

Wenn sich Coding-Conventions ändern, wird im Übersichts-Repository eine GitHub Action ausgeführt, die die aktualisierten .md-Dateien an Bibliotheken und Template-Projekte verteilt. Da die Ziel-Repositories in den Workflows explizit hinterlegt werden müssen, können fortlaufend neu entstehende Projekt-Repositories nicht direkt auf diesem Weg gepflegt werden. Deshalb werden die notwendigen Actions im Template-Projekt abgelegt. Alle daraus abgeleiteten Projekte übernehmen diese automatisch und können die aktuellen Conventions zentral aus dem Übersichts-Repository abrufen.

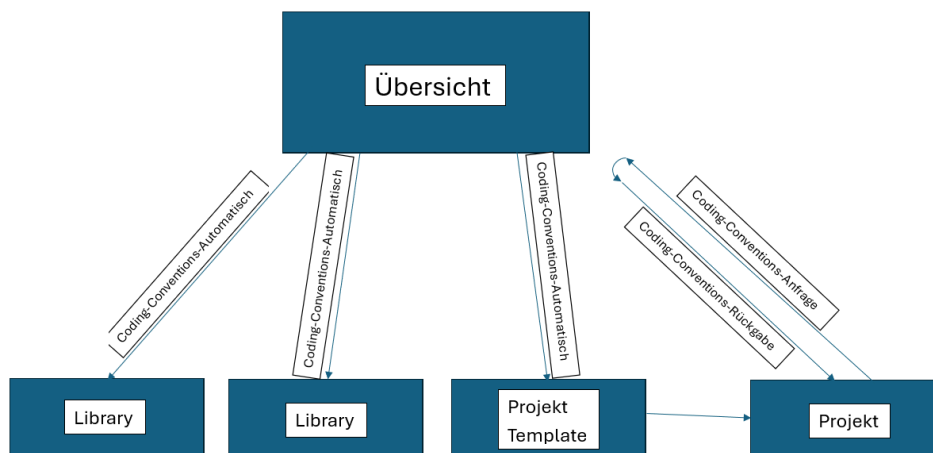


Abbildung 4 – Coding-Convention Verteilung

Diese Actions sind bereits in allen O-SW-Repositories vorhanden, die nicht als Projekt-Repositories geführt werden. Die Abläufe wurden intensiv getestet. Auch die automatische Übernahme in neu erzeugte Projekte ist bereits umgesetzt.

Zusätzlich wurde im Test-Repository implementiert, dass Copilot bei jeder Pull Request auf main automatisch alle enthaltenen Commits gegen die hinterlegten Coding-Conventions prüft und Verbesserungsvorschläge liefert. Um die Zuverlässigkeit realistisch zu bewerten, wurden bewusst typische Fehler eingebaut. Aus den resultierenden Copilot-Antworten konnten mehrere Problemfelder abgeleitet werden. Diese werden im Folgenden dargestellt, erläutert und hinsichtlich möglicher Lösungen bewertet.

4.2 Testablauf

Zur Bewertung der Zuverlässigkeit von GitHub Copilot wurde im Test-Repository ein separater Branch angelegt, in den gezielt typische Programmierfehler eingebracht wurden. Anschließend wurde jeweils eine Pull Request in main geöffnet, um die

konfigurierte Code-Review-Richtlinie auszulösen. Copilot analysierte die Änderungen und hinterließ entsprechende Kommentare.

Da Copilot auch die Pull-Request-Beschreibung und Diskussionen verarbeitet, wurde die Checkliste der absichtlich eingebauten Fehler jeweils erst nach Abschluss des Reviews ergänzt. So blieb die Auswertung nachvollziehbar und methodisch vergleichbar.

Wenn Anpassungen an Coding-Conventions oder Copilot-Anweisungen vorgenommen wurden, wurde die vorherige Pull Request geschlossen und eine neue erstellt. Damit konnte jede Änderung unter möglichst gleichen Rahmenbedingungen bewertet werden.

4.3 Ergebnisse und Problemanalyse

4.3.1 Grundproblem

In den ersten Copilot-Reviews zeigte sich, dass die Gründlichkeit der Befunde nicht ausreichte. Nach eigener Recherche und Rücksprache mit GitHub Copilot Chat wurde deutlich, dass die zugrunde liegenden Coding-Conventions vermutlich nicht präzise genug formuliert waren. Deshalb wurden die Kapitel „TwinCat“ und „Allgemein“ vollständig überarbeitet. Zur Unterstützung wurde der Microsoft-Copilot-Agent „Prompt Coach“ genutzt, um die Regeln im vorhandenen Kontext verständlicher und KI-kompatibler zu formulieren. Die Überarbeitung betraf vor allem die sprachliche Präzisierung sowie ergänzende Copilot-Anweisungen.

Die durch KI erzeugten Vorschläge wurden anschließend von mehreren Personen inhaltlich geprüft und nur nach sorgfältiger Bewertung übernommen. Eine ungeprüfte oder automatische Übernahme erfolgte ausdrücklich nicht.

Zusätzlich erhielt jede Regel eine eindeutige ID, auf die GitHub Copilot in Befunden verweisen kann. Die Struktur folgt dem Muster Kapitelnummer-Fehlernummer im Kapitel, beispielsweise „3.2.7-2“.

Nach diesen Anpassungen verbesserte sich die Qualität der Reviews deutlich. Dies zeigte sich unter anderem daran, dass Kapitel- und Regelreferenzen konsistent verwendet wurden und erkannte Probleme zumindest als unterdrückte Kommentare sichtbar wurden.

4.3.2 Problem Kommentarunterdrückung

Das zentrale Restproblem bestand darin, dass Copilot Benennungsregelverstöße für Variablentypen innerhalb von Structs nur eingeschränkt zuverlässig erkannte. Besonders betroffen war die Prüfung des Präfixes `stc_` für Struct-Typ-Variablen. In der Praxis wurden Verstöße zwar erkannt, die zugehörigen Kommentare jedoch häufig unterdrückt. Das Problem wurde gezielt reproduziert, indem im Testcode das erforderliche Präfix bei einem Variablentyp in einer Struct entfernt wurde. Zur Gegenmaßnahme wurden die relevanten Abschnitte der Coding-Conventions aus Abbildung 5 in den Kontextdialog übernommen und auf dieser Basis die in Listing 1 gezeigte Zusatzklausel formuliert.

****3.2.5-1 (Major)****
Der Datentyp JEDER Variable MUSS durch ein eindeutiges Präfix gekennzeichnet sein.

IEC-Datentyp	Präfix
BOOL	b_
BIT	bit_
BYTE	by_
WORD	w_
DWORD	dw_
LWORD	lw_
INT	i_
UINT	ui_
SINT	si_
USINT	usi_
DINT	di_
UDINT	udi_
LINT	li_
ULINT	uli_
REAL	r_
LREAL	lr_
TIME	t_
LTIME	lt_
DATE	d_
TIME_OF_DAY	tod_
STRING	str_
PVOID	pvo_
ARRAY	a_
STRUCT	stc_
ENUM	e_
UNION	u_
ANY	any_
REFERENCE TO	ref_
POINTER TO	ptr_

Abbildung 5 – Coding-Convention Variablen Präfixe

```
1 ### Zusatz: Referenzierung der Datentyp-Präfixe bei  
2 Benennungsverstößen (Mandatory)  
3  
4 Bei Benennungsverstößen, die Datentyp-Präfixe betreffen,  
5 MUSS als fachliche Grundlage **Tabelle 3.2.5 (Datentyp-Präfixe)**  
6 herangezogen werden.  
7  
8 Befunde MÜSSEN:  
9 - das **konkret erwartete Präfix** nennen (z.B. `b_`, `i_`, `di_`)  
10 - optional explizit auf **Tabelle 3.2.5** verweisen  
11  
12 Befunde DÜRFEN NICHT:  
13 - Datentypen semantisch interpretieren (z.B. „BOOL-Präfix“)  
14 - implizite Ableitungen ohne Tabellenreferenz enthalten  
15  
16 Zulässig:  
17 - „Feldname beginnt nicht mit dem erwarteten Präfix `b_` gemäß Tabelle 3.2.5.“  
18 - „Bezeichner beginnt nicht mit dem erforderlichen Präfix `e_` (siehe Tabelle  
19 3.2.5).“
```

Listing 1 – Klausel

Nach dieser Änderung erkannte GitHub Copilot die Regelverstöße zuverlässig. Die Hinweise wurden jedoch weiterhin teilweise als „suppressed due to uncertainty“ markiert. Zur Analyse wurde die GitHub-Copilot-Funktion „Agents“ genutzt. Diese kann das gesamte Repository inklusive früherer Pull Requests als Kontext einbeziehen und ermöglicht dadurch fundiertere Auswertungen. Die Analyse zeigte, dass GitHub die Kommentare selbst unterdrückte, sobald sie auf umbenannte Dateien verwiesen. Da Git-Umbenennungen im Diff häufig als Löschen und Neu-Anlegen dargestellt werden, werden solche Befunde schneller als unsicher eingestuft, weil:

- Die gesamte Datei als „neu hinzugefügt,“ erscheint
- GitHub keine stabile Diff-Kontext-Verknüpfung für umbenannte Dateien aufbauen kann
- Das Umbenennen an sich als intentionale Änderung gilt. Copilot widerspricht dem PR-Intent

Daraus ergibt sich: Copilot erkennt den Verstoß korrekt, GitHub unterdrückt den Kommentar jedoch im Anschluss.

Die durch den Agent gelieferten Analysen wurden manuell geprüft und mit den tatsächlichen Repository-Änderungen abgeglichen, um Fehlinterpretationen auszuschließen.

Als Gegenmaßnahme wurde vorgeschlagen, die in Listing 2 gezeigte Anweisung in die GitHub-Copilot-Instruktionen aufzunehmen.

```
1 ### Pflicht-Strategie für Violations in umbenannten DUT-Dateien  
2 (Mandatory)  
3 **3.2.12-2 (Mandatory Review Behavior)**  
4  
5 Wenn ein STRUCT- oder ENUM-Typ in einer **umbenannten** DUT-Datei  
6 deklariert wird  
7 (erkennbar daran, dass der alte Dateiname und der neue Dateiname im  
8 \.plcproj`-Diff  
9 sichtbar sind), MUSS Copilot den Typ-Benennungsverstoß auf **zwei** Arten  
10 melden:  
11  
12 1. An der Verwendungsstelle in einer **nicht umbenannten, modifizierten  
13 Datei**  
14 (z.B. `stc_FxControlPar.TcDUT`, `FxGroupManagement_V12.TcPOU`), sofern  
15 der  
16 Typ dort als neue Version des umbenannten Typs referenziert wird.  
17 Die Evidence MUSS die Zeile enthalten, die den Typnamen ohne korrektes Präfix  
18 zeigt.  
19 Der Befund MUSS explizit darauf hinweisen, dass die TYPE-Deklaration selbst  
verletzt ist.  
  
2. In der \.plcproj`-Datei an der Zeile, die den neuen (prefix-losen) DUT-  
Dateinamen  
enthält - sofern diese Zeile im Diff als geänderter `+`-Eintrag erscheint.  
  
**Copilot DARF NICHT** als einzigen Fundort die umbenannte DUT-Datei selbst  
angeben,  
da GitHub diese Kommentare als low-confidence unterdrückt.
```

Listing 2 – Anti suppression Klausel

Damit werden Verstöße nicht nur an der Deklaration, sondern zusätzlich an der ersten Verwendung in einer nicht umbenannten Datei gemeldet. Die Wahrscheinlichkeit einer Unterdrückung durch GitHub sinkt dadurch deutlich. Nach dem Test zeigte sich, dass die ursprünglichen Kommentare weiterhin teilweise unterdrückt wurden, Verstöße jedoch zusätzlich an der ersten Verwendung der betroffenen Variablen als Fehler erschienen. Da diese Fundstellen nicht in umbenannten Dateien lagen, blieben die entsprechenden Meldungen sichtbar.

4.4 Feedback und weitere Verbesserung

Nach der Umsetzung der Verbesserungen im Test-Repository wurde ein Abstimmungsgespräch mit Ralf Scheyerle geführt, der die Änderungen bewertete. Dabei merkte er an, dass die Instructions in Teilen zu eng gefasst seien. Dadurch funktioniere Copilot zwar zuverlässig als Reviewer, seine spätere Nutzung als Coding Agent werde jedoch unnötig eingeschränkt. Da gerade diese Flexibilität für den produktiven KI-Einsatz wichtig ist, wurde empfohlen, die Anweisungen gezielt zu öffnen, ohne die Review-Qualität zu verlieren.

Im Anschluss wurden die Instructions entsprechend überarbeitet. Konkret wurde die Klausel entfernt, die das Verhalten zu stark auf reines Reviewing begrenzte. Außerdem wurden die überarbeiteten TwinCat-Instructions in das Übersichts-Repository übernommen. Damit kann der in Abbildung 4 dargestellte Verteilungsprozess nun sowohl für Coding-Conventions als auch für Copilot-Instructions einheitlich genutzt werden.

Außerdem wurde angemerkt, dass Copilot bei Pull-Request-Reviews nur die veränderten Zeilen lesen kann. Dies führt dazu, dass bereits vorhandener Code nicht überprüft wird. Das zeigt sich vor allem darin, dass Fehler, die schon im Template-Projekt vorliegen, nicht korrekt erkannt werden. Hier musste daher eine Lösung gefunden werden, um Bestandscode zu prüfen.

Zu diesem Zweck wurde in Zusammenarbeit mit Ralf Scheyerle ein Agent entwickelt, der große Datenmengen komplexen Bestandscodes analysieren kann. Dies wurde dadurch ermöglicht, dass Ralf bereits einen Agenten besitzt, der qualitativ hochwertige Agent-Anweisungen formulieren kann. Mithilfe dessen konnte dieser Reviewer erfolgreich erstellt werden. Dieser wurde auch sofort eingesetzt, um die Template-Projekte und Standard-Libraries, soweit möglich, von Coding-Conventions-Verstößen zu bereinigen.

5

Pilotierung im Projekt

Um die praxisnahe Integration der eingeführten Quality-Assurance-Protokolle zu bewerten, wurden diese in ein Projekt eingebracht, das sich noch in der Vorinbetriebnahme befand. Dazu wurde mit den Projektprogrammierern David Kromer und Alexandru Trusi ein Termin angesetzt, in dem die geplanten Methoden gemeinsam besprochen wurden. Insbesondere die Branching-Strategien wurden positiv bewertet, da beide Programmierer die in Abschnitt 2.1 beschriebenen Herausforderungen bereits aus ihrem Arbeitsalltag kannten.

Alexandru brachte zusätzlich den Vorschlag ein, neben Feature-Banches pro Entwickler jeweils einen eigenen Develop-Branch zu nutzen. So kann zunächst unabhängig gearbeitet werden, bevor stabile Zwischenstände per Pull Request in develop überführt werden. Ergänzend können nutzerspezifische Feature-Banches für die Umsetzung konkreter Funktionen angelegt werden. Die Benennung könnte beispielsweise wie folgt aussehen:

- user/develop
- user/feature/NeuesFeature

Beide Programmierer signalisierten außerdem Offenheit gegenüber Copilot, sofern die Review-Ergebnisse konsistent und verlässlich bleiben.

In einer anschließenden Abstimmung zu den Meeting-Ergebnissen merkte Robin Hettel an, dass statt nutzerbezogener Branches eher anlagenbereichsbezogene Branches sinnvoll seien. Dadurch könnten Programmierer einzelne Bereiche zeitweise „reservieren“ und dort konzentriert arbeiten. Gleichzeitig würde dieser Ansatz helfen, keine gesonderten Richtlinien nur für die Anlagenprogrammierung einzuführen, sondern einen unternehmensweit einheitlichen Standard beizubehalten.

Nach der Formulierung dieser Richtlinien startete das Projekt in seine erste Phase, die virtuelle Inbetriebnahme. In dieser Phase werden die Roboter in einer Simulation angelegt und bereits grundlegend programmiert.

5.1 Umsetzung im Pilotprojekt

5.1.1 Branching Strategien

Die Branching-Strategie wurde auf Basis des angepassten Git-Flow-Modells umgesetzt. Dabei wurden die Aufgaben der einzelnen Branches klar abgegrenzt und der Merge-Prozess über Pull Requests verbindlich standardisiert. Ergänzend wurden Benennungsregeln für Feature-, Bugfix- und Release-Banches festgelegt, um eine hohe Nachvollziehbarkeit und einheitliche Abläufe sicherzustellen.

5.1.2 Copilot Code-Review

Das Copilot-gestützte Code-Review wurde analog zum Test-Repository in den Projektkontext überführt. Pull Requests auf main werden dabei automatisch gegen die hinterlegten Coding-Conventions geprüft. Die Ergebnisse fließen als strukturierte Review-Kommentare in den Freigabeprozess ein und unterstützen die frühzeitige Erkennung von Abweichungen.

Das Code-Review wird bei jedem Pull Request auf main ausgelöst, um die Einhaltung der Coding-Conventions systematisch zu prüfen.



Projektmanagementprozess mit ZenHub

ZenHub ist ein webbasiertes Projektmanagement-Werkzeug, das speziell auf Softwareentwicklungsprozesse zugeschnitten ist und sich direkt in GitHub integriert. Der zentrale Vorteil besteht darin, dass Aufgabenplanung und Quellcodeverwaltung in derselben Umgebung stattfinden. Dadurch entfallen Medienbrüche zwischen Ticket-System, E-Mail-Kommunikation und Repository-Verwaltung.

Die zugrunde liegenden Funktionsbausteine werden in der Herstellerdokumentation beschrieben [7].

Für die vorliegende Arbeit ist ZenHub vor allem deshalb relevant, weil neben der technischen Umsetzung auch die organisatorische Steuerung verbessert werden soll. Während Branching-Strategien und Code-Reviews primär die technische Qualität absichern, stärkt ZenHub die Prozessqualität: Priorisierung, Verantwortlichkeiten, Terminbezug und Transparenz über den Arbeitsfortschritt.

Funktional stellt ZenHub dafür mehrere Bausteine bereit:

- Boards zur operativen Steuerung der Aufgaben im Tagesgeschäft
- Epics zur Bündelung zusammenhängender Features oder Arbeitspakete
- Roadmaps zur zeitlichen Planung über mehrere Iterationen oder Meilensteine
- Automatisierungen für wiederkehrende Statuswechsel und Workflows

Insgesamt entsteht damit eine zentrale und nachvollziehbare Planungs- und Kommunikationsstruktur, die sowohl für kleine Teams als auch für bereichsübergreifende Zusammenarbeit geeignet ist.

6.1 Aktueller Stand bei E-SW

In der E-SW wird ZenHub aktuell primär für die Release-Planung der firmeneigenen Nutzeroberfläche FlexOp eingesetzt. Der Fokus liegt auf einer klaren Verbindung zwischen strategischer Planung (Roadmap und Epics) und operativer Umsetzung (Issues auf dem Board).

Der etablierte Ablauf in der E-SW lässt sich wie folgt zusammenfassen:

- Neue Aufgaben werden als Issues erfasst und inhaltlich vorqualifiziert.
- Der Teamleiter Simon Würzer priorisiert diese Issues nach Dringlichkeit und fachlicher Relevanz.
- Anschließend werden die priorisierten Tickets in den Bearbeitungsprozess überführt und passenden Teammitgliedern zugewiesen.
- Umfangreichere Vorhaben werden als Epic angelegt und in umsetzbare Einzel-Issues zerlegt.
- Für geplante Releases werden feste Zeitpunkte in der Roadmap hinterlegt und die zugehörigen Issues entsprechend zugeordnet.

Dieser Ablauf schafft hohe Transparenz über Zuständigkeiten und Fortschritt. Gleichzeitig wird sichergestellt, dass operative Aufgaben jederzeit im Kontext übergeordneter Release-Ziele stehen.

6.2 Übertragung auf die O-SW

Die Übertragung auf die O-SW verfolgt das Ziel, die bisher überwiegend mündliche und E-Mail-basierte Abstimmung durch einen standardisierten und nachvollziehbaren Prozess zu ersetzen. Insbesondere bei kurzfristigen Änderungen im Umfeld von FlexOp-Versionen ist eine zentrale Planung erforderlich, um Abhängigkeiten frühzeitig sichtbar zu machen und den Abstimmungsaufwand zu reduzieren.

Ausgehend von den in der E-SW etablierten Abläufen wird ZenHub in der O-SW so eingesetzt, dass strategische Planung und operative Umsetzung enger verknüpft werden. Aufgaben werden als Issues erfasst, priorisiert und den verantwortlichen Mitarbeitenden zugeordnet. Umfangreichere Vorhaben werden in Epics gebündelt und in umsetzbare Einheiten unterteilt. Für zeitkritische Arbeitspakete erfolgt zusätzlich eine Zuordnung zu geplanten Meilensteinen.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua quaerat voluptatem. Ut enim aequaleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si

aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos.

Durch dieses Vorgehen kann ZenHub nicht nur als reines Ticket-Tool genutzt werden, sondern als verbindendes Steuerungselement zwischen technischer Umsetzung, Branching-Prozess und Release-Kommunikation.



Schulung und Wissenssicherung

Damit die zuvor beschriebenen Methoden im Team wirksam und regelkonform angewendet werden, werden im nächsten Schritt strukturierte Schulungsunterlagen erarbeitet. Diese sollen einerseits eine grundlegende Einführung in Git und GitHub für neue Mitarbeitende und Auszubildende bieten und andererseits die neu eingeführten Prozesse und Richtlinien vermitteln.

Die Dokumentation verfolgt dabei zwei Ziele: Erstens soll das Onboarding neuer Teammitglieder systematisch unterstützt werden, zweitens soll ein abteilungsweit einheitlicher Wissensstand etabliert werden.

7.1 Aktuelles Wissen der Mitarbeiter

Zur bedarfsgerechten Ausgestaltung der Schulung wurde ein strukturierter Austausch mit dem Mitarbeiter Matthias Scheuring durchgeführt. Zum Zeitpunkt des Gesprächs verfügte er über geringe Vorerfahrung mit Git und GitHub, hatte jedoch bereits punktuelle Berührungspunkte in einem Projektkontext. Ziel des Termins war es, die aus Anwendersicht relevanten Einstiegshürden zu identifizieren und daraus priorisierte Schulungsinhalte abzuleiten.

Die inhaltlichen Ergebnisse werden im weiteren Projektverlauf noch vollständig konsolidiert. Als aktueller Arbeitsstand lassen sich folgende Schwerpunkte festhalten:

- Grundverständnis zu Git, GitHub und Versionshistorie
- Sicheres Arbeiten mit Branches, Pull Requests und Merges
- Orientierung im unternehmensspezifischen Ablaufmodell

In dem Gespräch mit Matthias nannte er einige Themen, welche ihm beim Einstieg in GitHub Probleme machten. Zunächst hatte er das Problem, dass er das Prinzip von Git nicht kannte und das Tool dessen nicht nutzte. Dadurch hatten andere Projektbeteiligte das Problem, dass sie ihn immer darum bitten mussten seinen Code zu commiten. Desweiteren kannte er zu Beginn die Mechanik der Branches in Git nicht, was die Folge hatte, dass er commits nur in Main vornahm. Das führte wie in Abschnitt 2.1.1 beschrieben zu Konflikten im Code und mit anderen Programmierern.

Auf Basis dieser Erhebung wurden die im Folgenden beschriebenen Schulungsbau-
steine zusammengestellt.

7.2 Wahl der Schulungsumgebung

Für die wirksame Vermittlung der Inhalte ist neben der fachlichen Aufbereitung auch die Auswahl geeigneter Lernumgebungen entscheidend. Die Bewertung erfolgte daher themenspezifisch.

7.2.1 Microsoft Learn

Microsoft Learn bietet etablierte Einstiegskurse zur Arbeit mit GitHub. Für den vorliegenden Anwendungsfall ist die Plattform jedoch nur eingeschränkt geeignet, da die Inhalte nicht an unternehmensspezifische Prozesse angepasst werden können. Zudem enthalten die Kurse teilweise Vorgehensweisen, die in der [fpt Systems GmbH](#) nicht eingesetzt werden.

7.2.2 GitHub Skills

GitHub Skills stellt einen qualitativ hochwertigen Kurs bereit, der den Einstieg in GitHub und die dortigen Workspaces unterstützt. Im Rahmen einer Erprobung zeigte sich jedoch, dass der Schwerpunkt stark auf Workspace-basierten Szenarien liegt. Da in der [O-SW](#) überwiegend mit [TwinCat](#) und [WorkVisual](#) gearbeitet wird, ist die direkte Übertragbarkeit auf den Zielkontext begrenzt.

7.2.3 YouTube

Auf YouTube sind zahlreiche didaktisch gut aufbereitete Inhalte verfügbar, die Funktionen und Nutzung von Git und GitHub umfassend erläutern. Da diese Inhalte die unternehmensspezifischen Prozesse naturgemäß nicht abbilden, eignen sie sich nicht

als alleinige Schulungsgrundlage. Als ergänzendes Einstiegsmedium sind sie jedoch sinnvoll.

7.2.4 Eigene Dokumentation

In der [fpt Systems GmbH](#) existiert bereits ein HTML-basiertes Intranet-Wiki zur zentralen Bereitstellung von Wissensinhalten. Da die Schulungsunterlagen dort eigenständig erstellt und gepflegt werden können, ist diese Lösung besonders geeignet, um den unternehmensspezifischen Umgang mit Git und GitHub abzubilden. Zusätzlich können multimediale Ergänzungen wie Abbildungen und externe Verweise integriert werden. Dadurch lassen sich die in Abschnitt 7.2.3 genannten Videos gezielt als ergänzende Lernressourcen einbinden.

7.3 Umsetzung der Dokumentation

7.3.1 Git und GitHub Grundlagen

Die Dokumentation der Git- und GitHub-Grundlagen bildet den Einstieg für neue Mitarbeitende und Auszubildende. Behandelt werden die zentralen Konzepte der Versionsverwaltung, die Arbeit mit Branches sowie der Ablauf von Pull Requests und Reviews im Unternehmenskontext. Die Inhalte sind so aufgebaut, dass sie sowohl für das Onboarding als auch als Nachschlagewerk im laufenden Projektbetrieb genutzt werden können.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor
 incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequae
 doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si
 aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet
 transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri ampli-
 ficarique non possit. At etiam Athenis, ut e patre.

7.3.2 Branching Strategien

Im Schulungsbaustein zu Branching-Strategien wird das in dieser Arbeit eingeführte Vorgehen praxisnah vermittelt. Der Schwerpunkt liegt auf der eindeutigen Zuordnung von Branch-Typen, der strukturierten Nutzung von Pull Requests sowie der nachvollziehbaren Integration in develop und main. Ziel ist eine konsistente Anwendung der festgelegten Regeln im Tagesgeschäft.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At.



Fazit

Im Rahmen dieser Arbeit wurden für die identifizierten Problemfelder Branching, Management und Code-Review konkrete Maßnahmen konzipiert und in den Projektkontext überführt. Die Einführung eines angepassten Git-Flow-Modells, der Einsatz von Copilot-gestützten Reviews sowie die Einbindung von ZenHub adressieren sowohl technische als auch organisatorische Schwachstellen der bisherigen Arbeitsweise.

Die Einordnung der erwarteten Prozesswirkung kann ergänzend auf etablierten Befunden zu Delivery- und Team-Performance gestützt werden [8].

Die durchgeführten Tests und Abstimmungen zeigen, dass die gewählten Ansätze grundsätzlich geeignet sind, Transparenz, Nachvollziehbarkeit und Qualität in der Zusammenarbeit zu erhöhen. Gleichzeitig wurde deutlich, dass die Wirksamkeit der Maßnahmen von klar formulierten Richtlinien, konsistenter Anwendung im Team und einer belastbaren Schulungs- und Dokumentationsbasis abhängt.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificare non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et.

Insgesamt legt die Arbeit damit eine praktikable Grundlage für einen einheitlichen, GitHub-basierten Entwicklungs- und Steuerungsprozess in der [fpt Systems GmbH](#).

A Literatur

- [1] G. Gousios, M. Pinzger, und A. van Deursen, „An Exploratory Study of the Pull-based Software Development Model“, in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, 2014, S. 345–355. doi: [10.1145/2568225.2568260](https://doi.org/10.1145/2568225.2568260).
- [2] V. Driessen, „A successful Git branching model“. [Online]. Verfügbar unter: <https://nvie.com/posts/a-successful-git-branching-model/>
- [3] A. Bacchelli und C. Bird, „Expectations, Outcomes, and Challenges of Modern Code Review“, in *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, 2013, S. 712–721. doi: [10.1109/ICSE.2013.6606617](https://doi.org/10.1109/ICSE.2013.6606617).
- [4] S. McIntosh, Y. Kamei, B. Adams, und A. E. Hassan, „The Impact of Code Review Coverage and Code Review Participation on Software Quality“, *Empirical Software Engineering*, Bd. 21, Nr. 5, S. 1863–1905, 2016, doi: [10.1007/s10664-015-9388-4](https://doi.org/10.1007/s10664-015-9388-4).
- [5] GitHub, Inc., „About pull request reviews“. [Online]. Verfügbar unter: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/code-quality-features/about-pull-request-reviews>
- [6] GitHub, Inc., „About protected branches“. [Online]. Verfügbar unter: <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches>
- [7] ZenHub Technologies Ltd., „ZenHub Help Center“. [Online]. Verfügbar unter: <https://help.zenhub.com/>
- [8] N. Forsgren, J. Humble, und G. Kim, *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.

B Glossar

E-SW: Entwicklung Software - Abteilung bei der fpt Systems GmbH, welche die Entwicklung neuer FlexOp Versionen und anderer Hochsprachenprojekten übernimmt [2](#), [5](#), [6](#), [7](#), [7](#), [7](#), [13](#), [13](#), [26](#), [26](#), [26](#), [26](#)

FlexOp: Firmeneigene Nutzeroberfläche zu Steuerung und Verwaltung von Anlagen [6](#), [7](#), [26](#), [26](#)

O-SW: Operations Software - Abteilung bei der fpt Systems GmbH, welche die Programmierung von SPS Projekten übernimmt [2](#), [5](#), [5](#), [6](#), [6](#), [6](#), [7](#), [7](#), [7](#), [7](#), [10](#), [11](#), [12](#), [17](#), [26](#), [26](#), [26](#), [29](#)

TwinCat: Aktuelle Version der Entwicklungsumgebung für Beckhoff SPS [6](#), [18](#), [29](#)

WorkVisual: Engineering-Umgebung zur Programmierung und Konfiguration von KU-
KA Robotern. [29](#)

fpt Systems GmbH: Tochterfirma der FPT Robotik GmbH & Co. KG, welche für die Softwareentwicklung verantwortlich ist. [7](#), [29](#), [30](#), [32](#)

Sperrvermerk

Die vorliegende Arbeit mit dem Titel

Konzeption und Implementierung eines GitHub-basierten Branching-, Review- und Projektmanagementprozesses mit KI-gestützter Code-Compliance

enthält unternehmensinterne bzw. vertrauliche Informationen der fpt Systems GmbH, ist deshalb mit einem Sperrvermerk versehen und wird ausschließlich zu Prüfungszwecken im Studiengang Elektrotechnik der Duale Hochschule Baden-Württemberg Ravensburg Campus Friedrichshafen vorgelegt.

Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungsprozesses und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anders lautende Genehmigung der Ausbildungsstätte (fpt Systems GmbH) vorliegt.

Amtzell 21.05.2026



Florian Germann

Selbstständigkeitserklärung

Gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden-Württemberg vom 29.09.2017. Ich versichere hiermit, dass ich meine Arbeit mit dem Thema:

Konzeption und Implementierung eines GitHub-basierten Branching-, Review- und Projektmanagementprozesses mit KI-gestützter Code-Compliance

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass alle eingereichten Fassungen übereinstimmen.

Amtzell 21.05.2026



Florian Germann